



DEPARTMENT OF COMPUTER ENGINEERING

**FACULTY OF ENGINEERING
UNIVERSITY OF RUHUNA**

EC8207 – Applied Big Data Engineering

Mini Project Report

GAWESH L. A. M. S.

EG/2020/3941

RANAWAKA N. L. N.

EG/2020/4143

Contents

1	Introduction	2
2	Architecture Overview	2
3	Justification of Tools Selected	3
3.1	Transaction Generator (Python Producer)	3
3.2	Apache Kafka - Ingestion Layer	4
3.3	Apache Spark Structured Streaming - Processing Layer	5
3.4	Parquet Data Lake - Storage Layer	6
3.5	Apache Airflow - Orchestration Layer	6
4	Event Time vs. Processing Time	7
5	Ethics and Privacy Implications	8
5.1	Privacy Concerns	8
5.2	Data Governance Measures	8
5.3	Ethical Use of Fraud Flags	9
6	Conclusion	9

1 Introduction

This report documents the design and implementation of a real-time fraud detection pipeline built for a digital wallet provider. The system ingests credit card transactions, detects fraud in real-time using two rules (high-value transactions and impossible travel), stores clean data in a Parquet-based data lake, and generates reconciliation and analytics reports every 6 hours using Apache Airflow.

The pipeline follows a **Lambda Architecture** pattern, where there is a speed layer for real-time fraud detection and a batch layer for periodic reporting and reconciliation. The rest of this report explains why each tool was selected, how the data flows through the system, the approach taken to handle semantics, and the ethical considerations around processing financial transaction data.

2 Architecture Overview

We chose the **Lambda Architecture** for this project because the scenario has two clearly different requirements that run at the same time:

1. **Real-time fraud detection** — Transactions need to be checked immediately as they come in. For example, if someone spends \$10,000 from two different countries in 5 minutes, we cannot wait 6 hours to catch that. The speed layer handles this.
2. **Batch reporting and reconciliation** — Every 6 hours, the system needs to read all the stored data, compare total ingress amounts vs. validated amounts, and generate reports. This is a batch workload that does not need to run in real-time.

Lambda fits this naturally because it separates these two concerns. The speed layer (Kafka + Spark Streaming) handles immediate processing, while the batch layer (Airflow + Spark batch reads on Parquet) handles the periodic reporting. Both layers write to and read from the same Parquet data lake, which acts as the shared serving layer.

We considered Kappa Architecture (streaming only, no separate batch layer), but it would have been overkill for the reporting side. The reconciliation report just needs to read stored Parquet files and do some aggregations and there is no need to re-stream historical data for that. Lambda keeps things simple and cleanly separated.

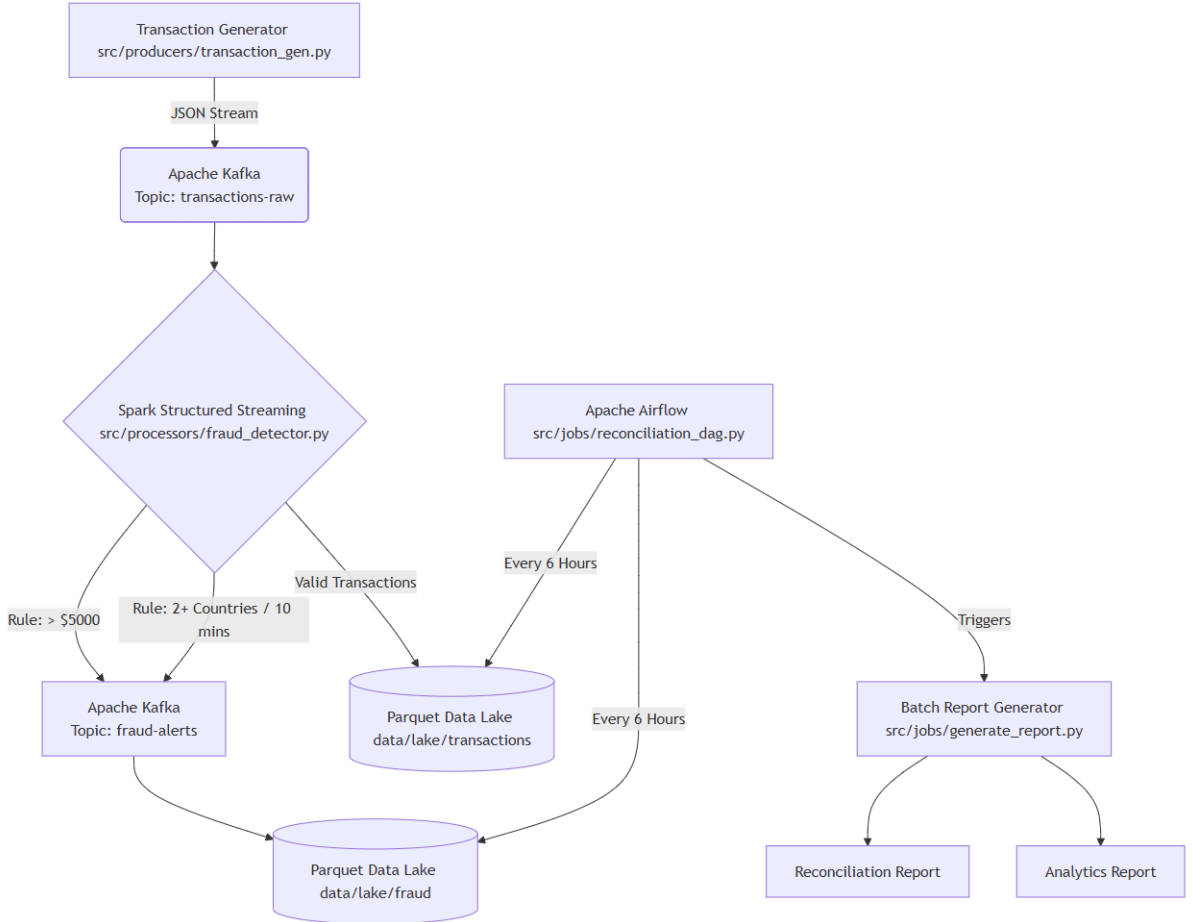


Figure 1: End-to-end architecture of the FinTech Fraud Detection Pipeline

3 Justification of Tools Selected

3.1 Transaction Generator (Python Producer)

The data source is a Python script (`src/producers/transaction_gen.py`) that simulates credit card transactions. Each transaction contains: `user_id`, `timestamp`, `merchant_category`, `amount`, and `location` (country).

The generator uses **controlled random probabilities** to make sure the output matches what the assignment asks for. Here is how:

- **Normal transactions** make up roughly 85% of the output. These have amounts between \$10 and \$4,500 (below the \$5,000 threshold). For normal transactions, 85% are domestic and 15% come from a randomly selected foreign branch country.
- **High-value fraud** is injected with a combined 10% probability — split equally between high-value foreign (5%) and high-value domestic (5%). When triggered, the generator picks an amount between \$5,001 and \$15,000, which will trigger the `HIGH_VALUE` rule downstream. The foreign variant also places the transaction in a

random foreign country, meaning it can contribute to impossible travel patterns if paired with other transactions.

- **Impossible travel fraud** is injected with a 5% probability. When triggered, the generator creates a *new user* and generates two transactions: one from a random country, and a second from a *different* country with a timestamp offset of 1–5 minutes (well within the 10-minute detection window). Both transactions share the same `user_id`, guaranteeing the `IMPOSSIBLE_TRAVEL` rule will fire in Spark.
- **Combined fraud** — the high-value foreign transactions (5%) inherently combine both a high amount and a foreign location. While impossible travel transactions use normal amounts by default, the system does not prevent a high-value foreign transaction from appearing near another transaction from the same user, so overlap can occur naturally.

The merchant categories are randomly selected from a predefined list of six: Food, Electronics, Travel, Fashion, Utilities, and Healthcare. This randomness ensures the analytics report (Fraud by Merchant Category) has a natural distribution rather than being concentrated in one category.

The key design choice here is that fraud is *occasional but guaranteed*. If we made fraud purely random with very low probability, we might run the pipeline for an hour and see zero fraud, which makes testing and demonstration difficult. The controlled probabilities ensure we always get a reasonable mix.

3.2 Apache Kafka - Ingestion Layer

Kafka was chosen as the ingestion layer for several reasons:

- **Decoupling:** The producer and consumer (Spark) do not need to know about each other. The generator just pushes to a Kafka topic, and Spark just reads from it. If Spark goes down for a minute, Kafka retains the messages and Spark can catch up when it restarts.
- **Durability:** Kafka persists messages to disk with configurable retention. This means if the Spark job crashes mid-processing, no transactions are lost.
- **Scalability:** If we ever needed to scale this to handle millions of transactions per second, Kafka can handle it by adding more brokers and partitions.
- **Ecosystem fit:** Spark Structured Streaming has first-class support for Kafka as a source. The integration is straightforward and well-documented.

Why 3 partitions? The `transactions-raw` topic is configured with 3 partitions. The reasoning is:

1. With 3 partitions, Spark can read from up to 3 parallel tasks, which gives decent parallelism for our workload without overcomplicating things.
2. The partition key is `user_id`. This is important because the impossible travel rule needs to compare consecutive transactions from the *same user*. By partitioning on `user_id`, all transactions for a given user go to the same partition, which means they arrive in order at the same Spark task. This avoids the complexity of cross-partition state management.
3. Three partitions is a reasonable default for a development/demo setup. In production, this would be tuned based on throughput requirements and consumer parallelism.

A separate topic `fraud-alerts` (also with 3 partitions) receives flagged transactions. This separation ensures downstream consumers (like a security dashboard or notification service) can subscribe only to confirmed fraud without filtering through all traffic.

3.3 Apache Spark Structured Streaming - Processing Layer

Spark Structured Streaming was chosen over Apache Storm for the following reasons:

- **Unified API:** Spark uses the same DataFrame API for both streaming and batch processing. This means the fraud detection logic and the batch report generation use similar code patterns, which reduces the learning curve and maintenance burden.
- **Stateful processing:** The impossible travel rule requires maintaining state, specifically, the last known country and timestamp for each user. Spark Structured Streaming supports stateful operations natively through windowed aggregations and `groupBy` operations with watermarks.
- **Exactly-once semantics:** With checkpointing enabled (stored in `data/checkpoints/`), Spark guarantees that each transaction is processed exactly once, even if the job restarts. This is critical for a financial system where double-counting a fraud alert or missing one is unacceptable.
- **Parquet integration:** Spark reads and writes Parquet natively and efficiently. Since our data lake uses Parquet, this is a natural fit.

The fraud detector (`src/processors/fraud_detector.py`) applies two rules in sequence:

1. **HIGH_VALUE:** Any transaction with `amount > 5000` is immediately flagged. This is a stateless check. It only looks at the current transaction.
2. **IMPOSSIBLE_TRAVEL:** For each incoming transaction, the processor checks the user's last known location and timestamp (maintained in Spark's state store).

If the user was in a different country less than 10 minutes ago, it is flagged. This is a stateful check.

After processing, transactions are written to two sinks:

- **Clean transactions** go to `data/lake/transactions/` as Parquet files.
- **Fraud transactions** go to `data/lake/fraud/` as Parquet files and are also pushed to the `fraud-alerts` Kafka topic for real-time consumption.

3.4 Parquet Data Lake - Storage Layer

Parquet was selected over alternatives like CSV, JSON, or a relational database for several reasons:

- **Columnar format:** The batch reports need to aggregate on specific columns (e.g., sum of amounts, count by category). Columnar storage means Spark only reads the columns it needs, which is much faster than row-based formats.
- **Compression:** Parquet compresses data very efficiently.
- **Schema enforcement:** Parquet files carry their schema with them. This prevents issues like a field accidentally being written as a string in one batch and an integer in another.
- **No infrastructure overhead:** Unlike PostgreSQL or Cassandra, Parquet files do not require running a separate database server. For a project of this scale, file-based storage is simpler and sufficient.

The data lake is organized into two directories: `data/lake/transactions/` for validated (clean) data and `data/lake/fraud/` for flagged transactions. This separation makes the batch layer's job straightforward — the reconciliation report reads from both directories and compares totals.

3.5 Apache Airflow - Orchestration Layer

Airflow orchestrates the batch reporting pipeline through a DAG (`src/jobs/reconciliation_dag.py`) scheduled to run every 6 hours. The DAG triggers `src/jobs/generate_report.py`, which:

1. Reads all Parquet files from `data/lake/transactions/` and `data/lake/fraud/`.
2. Calculates total ingress count and amount (clean + fraud combined).
3. Calculates validated count and amount (clean only).
4. Generates the **Reconciliation Report** comparing these figures.

5. Aggregates fraud counts by merchant category to generate the **Analytics Report**.
6. Writes both reports to `data/reports/`.

Airflow was chosen because:

- It provides a web UI (accessible at `localhost:8081`) where we can monitor DAG runs, check logs, and manually trigger runs for testing.
- It handles retries and failure notifications, which is important for a scheduled financial report.
- The DAG definition is just Python code, so it integrates naturally with the rest of the project.

4 Event Time vs. Processing Time

This distinction is one of the most important aspects of the pipeline’s correctness.

- **Event Time:** The timestamp embedded in the transaction by the generator. This represents when the transaction *actually happened* at the point of sale.
- **Processing Time:** The time when Spark receives and processes the transaction. This can be seconds or even minutes after the event time due to network delays, Kafka buffering, or Spark micro-batch intervals.

Why this matters for impossible travel: Consider this scenario. User A makes a transaction in Germany at 10:00:00 (event time) and another in Japan at 10:08:00 (event time). Due to network delay, the Japan transaction arrives at Spark first (processing time 10:08:15) and the Germany transaction arrives second (processing time 10:08:45). If we used processing time, the system would see Germany *after* Japan and might calculate the wrong time difference, or worse, set the “last known location” incorrectly.

To handle this, our Spark processor uses **Event Time** as the primary time reference. The transaction’s embedded `timestamp` field is used for all window calculations and state management. Additionally, **Watermarking** is configured with a threshold (e.g., 10 minutes). This tells Spark: “Keep the state window open for an extra 2 minutes to account for late-arriving data.” After the watermark passes, Spark safely discards old state to prevent memory from growing indefinitely.

This ensures that even if transactions arrive out of order, the impossible travel rule still evaluates correctly based on when the transactions actually happened, not when Spark received them.

For the **batch layer**, event time is also used. The reconciliation report aggregates trans-

actions based on their event timestamps, not when they were written to Parquet. This ensures the 6-hour window in the report corresponds to 6 hours of actual transaction activity.

5 Ethics and Privacy Implications

5.1 Privacy Concerns

A fraud detection pipeline like this processes extremely sensitive data. Each transaction record contains:

- **User identity** (`user_id`) — can be linked to a real person.
- **Location data** (`country`) — reveals where a person is at a given time.
- **Spending behavior** (`amount`, `merchant_category`) — reveals lifestyle, habits, and financial status.
- **Temporal patterns** (`timestamp`) — reveals daily routines and travel patterns.

Combined, this data creates a detailed behavioral profile of every user. Even though the purpose is fraud detection, the same data could be misused for:

- **Unauthorized surveillance:** Tracking a user's physical movements across countries and time.
- **Discriminatory profiling:** If the fraud model disproportionately flags transactions from certain countries or merchant categories, it could unfairly target specific demographics.
- **Data breaches:** If the Parquet data lake or Kafka topics are accessed by unauthorized parties, all user financial data is exposed.

5.2 Data Governance Measures

To address these concerns, the following governance practices should be applied:

1. **Data Minimization:** Only collect and store the fields strictly necessary for fraud detection. For example, exact GPS coordinates are not needed — country-level location is sufficient for the impossible travel rule.
2. **Pseudonymization:** The `user_id` should be a hashed or pseudonymized identifier, not a real name or account number. This way, even if the data lake is compromised, individual users cannot be directly identified.

3. **Access Control:** The Kafka topics and Parquet data lake should be protected with role-based access control (RBAC). Only the fraud detection service and the Airflow batch job should have read access to transaction data.
4. **Retention Policies:** Transaction data should not be stored indefinitely. A retention policy (e.g., 90 days) should be enforced, after which data is deleted or anonymized.
5. **Audit Logging:** All access to the data lake and fraud alerts should be logged. If someone queries user transaction history, there should be a record of who accessed it and why.
6. **Regulatory Compliance:** In a production setting, this system would need to comply with regulations like GDPR (if operating in Europe) or PCI-DSS (for credit card data). GDPR gives users the right to request deletion of their data, which means the pipeline must support selective deletion from the Parquet data lake.
7. **Bias Monitoring:** The fraud detection rules should be periodically reviewed to check whether they disproportionately flag certain user groups. For example, if users who frequently travel internationally are being flagged at higher rates, the impossible travel threshold might need adjustment.

5.3 Ethical Use of Fraud Flags

When a transaction is flagged as fraud, it should not automatically result in account suspension or denial. False positives could occur. A legitimate user might genuinely travel from one country to another and make a large purchase. The fraud flag should trigger a *review process*, not an automated punishment. The system should be designed to assist human analysts, not replace their judgment entirely.

6 Conclusion

This project demonstrates a working end-to-end data pipeline that satisfies both real-time and batch processing requirements for a FinTech fraud detection scenario. The Lambda Architecture cleanly separates the real-time fraud detection (Kafka + Spark Streaming) from the batch reporting layer (Airflow + Parquet), while sharing a common data lake. The controlled transaction generator ensures reproducible fraud patterns for testing, and the use of event time with watermarking ensures correctness even when transactions arrive out of order. The ethical considerations highlight that while fraud detection is necessary, it must be implemented with strong data governance to protect user privacy and prevent misuse.